

Cours sur Aria, ses rôles, aria-label, aria-labelledby et aria-hidden

Complément au HTML sémantique (utilisation de balises non pas uniquement pour leurs fonctionnalités, mais aussi pour leur sens : `<sidebar>`, `<section>`), ARIA permet de renforcer l'accessibilité d'une application web pour les personnes en situation de handicap. Bien que crucial pour le bon fonctionnement des technologies d'assistance, l'usage d'ARIA peut paraître cryptique pour les intégrateurs non-initiés. Voici quelques repères pour bien commencer.

Qu'est-ce qu'ARIA ?

ARIA, acronyme pour Accessible Rich Internet Applications, est un ensemble d'attributs en HTML qui permettent d'indiquer l'utilité ou l'état d'un élément aux technologies d'assistance. ARIA est conçu pour fonctionner avec ces dernières afin d'aider l'utilisateur à mieux percevoir la page. Utilisé avec un HTML sémantique et solide, il permet de renforcer l'accessibilité des sites.

Ci-dessous, quelques attributs ARIA importants avec leurs bonnes pratiques associées.

Quelques attributs ARIA importants

Les rôles

Les rôles d'ARIA permettent d'ajouter une précision sémantique à un élément HTML afin que sa nature soit mieux reconnue par les technologies d'assistance comme les lecteurs d'écran mais aussi les bots d'indexation (un programme automatisé ou un bot qui recherche systématiquement les sites Web et indexe leur contenu). Dans les faits, ils permettent donc d'écrire du HTML sémantique d'une autre façon qu'en utilisant une balise sémantique HTML5 comme `<button>` ou `<nav>`.

Ainsi, il est possible d'indiquer au navigateur et à un lecteur d'écran que nous créons un bouton à l'aide de la balise sémantique :

```
<button>Se connecter</button>
```

Mais aussi avec une balise "div" grâce au rôle "button" :

```
<div role="button">Se connecter</div>
```

Un lecteur d'écran reconnaîtra chacun de ces deux éléments comme un bouton. Quelle option choisir, alors ?

Avant toute chose, une petite explication s'impose.

La deuxième option a beau être similaire au niveau sémantique, il ne s'agit pas d'un bouton au niveau fonctionnel : il va falloir lui donner des styles pour tous ses états, les événements en JavaScript, faire en sorte qu'il gère le focus, que

le curseur change au survol... Alors que tout ça est déjà pris en charge par la balise HTML `<button>`.

En d'autres termes, utiliser la deuxième option revient à réinventer la roue. Avec la première, on a une base de logique et de styles sur laquelle vous allez pouvoir construire votre composant.

Dès lors que le HTML5 prévoit une balise dédiée et bien supportée par les navigateurs récents, il est fortement conseillé de s'en servir.

On optera donc pour la balise `<button>` dans l'écrasante majorité des cas.

Pour un deuxième exemple, écrire `<h2 role="presentation">Les Licornes sont gentilles</h2>` supprime les sémantiques de titre de l'élément `h2`, le rendant équivalent à `<div>Les Licornes sont gentilles</div>`. Les sémantiques de rôle de titre sont supprimées, mais le contenu lui-même est toujours disponible.

Mais quand est-il préférable d'utiliser un rôle ARIA plutôt qu'une balise sémantique ?

Nous venons de voir qu'il était préférable d'utiliser les balises sémantiques. Or, il n'en existe pas encore pour tous les rôles ARIA possibles.

D'autres balises ne sont pas encore bien supportées par tous les navigateurs, par exemple la balise `<dialog>` qui n'est pas encore totalement supportée mais pourtant nécessaire à la bonne utilisation des modales par les utilisateurs malvoyants. Dans ce cas-là, il sera possible d'utiliser le rôle `dialog` sur une balise faiblement sémantique :

```
<div role="dialog" aria-label="Confirmation d'annulation" aria-hidden="false"><!--  
Le contenu de votre modale --></div>
```

Voire même de l'utiliser conjointement à la balise pour en augmenter le support :

```
<dialog role="dialog" aria-label="Confirmation d'annulation"  
aria-hidden="false"><!-- Le contenu de votre modale --></dialog>
```

Les labels ARIA

L'attribut **aria-label** permet d'ajouter un nom accessible qui ne sera visible que par les technologies d'assistance. La valeur de cet attribut sera par exemple dictée par le lecteur d'écran au survol ou au focus de l'élément. Il est particulièrement employé sur les éléments interactifs ne décrivant pas leur fonction comme les burger buttons. Sa sémantique étant proche de la balise `<label>`, il convient de l'utiliser lorsqu'il n'est pas possible d'utiliser cette dernière.

L'utilité de l'attribut `aria-label` n'étant pas toujours claire, voici un petit diagramme qui permet de déterminer si on a besoin d'utiliser `aria-label` sur un élément selon son utilité.

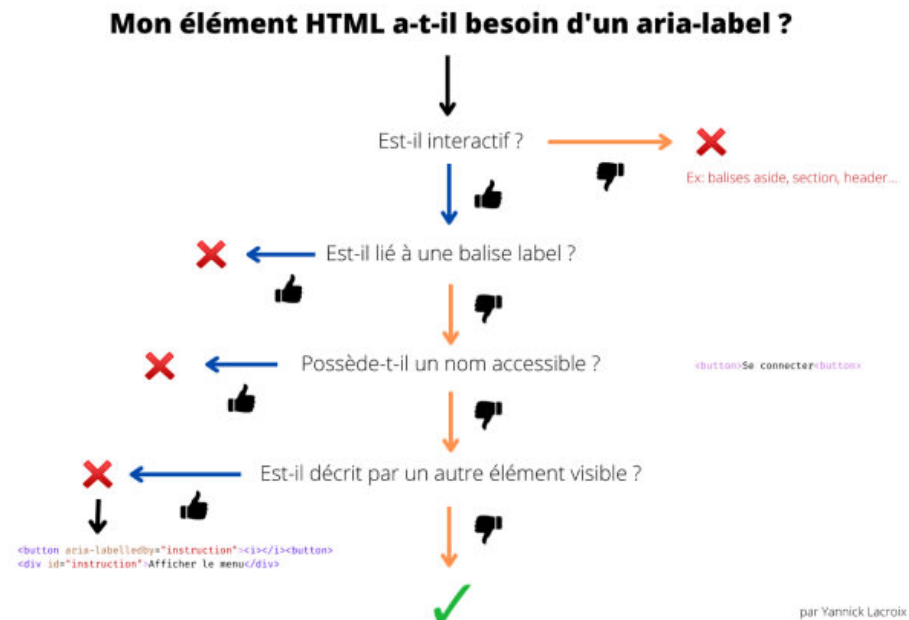


Figure 1: aria-label

A la lecture de ce diagramme, on remarque que cet attribut n'est nécessaire que dans des situations très spécifiques. Un des concepts clés d'ARIA est le suivant :

No ARIA is better than bad ARIA

Si on ajoute des aria-label aux éléments contenant déjà un nom accessible, l'information deviendra vite redondante et l'utilisateur de lecteur d'écran sera rapidement submergé d'informations sonores.

Par exemple, pour l'élément suivant :

```
<button aria-label="bouton réinitialiser">Réinitialiser</button>
```

Un logiciel comme NVDA (NonVisual Desktop Access) lira deux fois “bouton” et “réinitialiser” au survol ou au focus. Ce qui peut facilement gêner l'utilisateur n'utilisant que le son pour naviguer sur l'application.

Quelques astuces pour bien utiliser les labels aria

- La valeur de l'attribut doit être concise et ne pas reprendre le nom de la balise.
- La valeur doit être écrite dans la langue du site sauf exception.
- Bien vérifier bien d'en avoir réellement besoin et de ne pas disposer d'autre alternative avant de l'utiliser.

- Privilégier l'attribut **aria-labelledby** si l'élément est décrit ailleurs sur la page.

L'attribut “ARIA labelledby”

L'attribut **aria-labelledby** identifie l'élément (ou les éléments) qui servent de libellé à l'élément sur lequel il est appliqué.

Par exemple :

```
<span
  role="checkbox"
  aria-checked="false"
  tabindex="0"
  aria-labelledby="tac"></span>
<span id="tac">J'accepte les conditions générales.</span>
```

Ici, le rôle **checkbox** est destiné aux contrôles interactifs cochables. Les éléments contenant **role="checkbox"** doivent également inclure l'attribut **aria-checked** afin d'exposer l'état de la case à cocher aux technologies d'assistance. L'élément peut être atteint via la touche Tab, et l'ordre de focus suit le DOM.

*Le **** avec **role="checkbox"** et **aria-labelledby** est à utiliser seulement si on a un design ultra personnalisé où un input natif ne peut pas être stylé correctement.*

L'attribut “ARIA hidden”

Contrairement aux deux attributs précédents, **aria-hidden** ne permet pas d'ajouter de la sémantique à un élément mais au contraire de cacher des éléments du DOM aux yeux des technologies d'assistance. L'élément ne sera pas retiré mais il ne sera plus descriptible par un lecteur d'écran, par exemple.

Cet attribut nous permet dans un premier temps de veiller à la bonne cohérence entre affichage et sémantique : si un élément n'est plus affiché, il ne doit plus être détecté par les technologies d'assistance.

aria-hidden ne doit pas être utilisé sur les éléments visibles à l'écran, sauf si cela améliore l'expérience de l'utilisateur de technologies d'assistances.

Par exemple, bon nombre d'applications floutent ou grisent l'arrière-plan lorsqu'une modale apparaît. Souvent même, les événements d'arrière-plan seront désactivés. Ce qui est bien, mais pas assez... Les technologies d'assistance continueront à détecter l'arrière-plan, et donc à lire son contenu. Difficile pour une personne malvoyante (qui n'utilise pas ces indices visuels pour se repérer sur notre application) de comprendre que l'élément important de la page est devenu la modale ; on va donc appliquer **aria-hidden="true"** à notre arrière plan : seule notre modale sera alors lisible par les technologies d'assistance. L'utilisateur saura alors où il se trouve sur notre page.

Quelques astuces pour bien utiliser l'attribut `aria-hidden`

- S'assurer de la cohérence entre affichage et sémantique : appliquer cet attribut aux éléments portant les propriétés CSS `display:none` ou `opacity:0` qui cachent l'élément à l'œil nu sans les retirer du DOM.
- Sur ReactJS ou Vue.js, pas besoin d'appliquer d'`aria-hidden` sur un élément dont l'affichage est conditionné vu qu'il sera directement retiré du DOM au rechargement du composant.
- Ne surtout pas l'utiliser sur la balise `<body>` !

Commencer avec ARIA

ARIA est un atout de taille pour renforcer la sémantique de son application et donc de la rendre plus accessible. Cependant, il n'est pas forcément évident de savoir par où commencer pour s'en servir.

Se focaliser sur la sémantique

Il n'est pas nécessaire de connaître tous les attributs ARIA pour commencer à utiliser ARIA. Dans un premier temps, il faut se concentrer sur la sémantique de l'application, c'est-à-dire sur le HTML (certaines balises un peu mal aimées comme `<fieldset>` ou `<aside>` méritent plus d'attention) et sur les rôles. En utilisant du HTML sémantique, on utilise déjà des rôles sans le savoir.

Pour chaque élément créé, il faut s'assurer que celui-ci communique d'abord clairement sa nature au navigateur. Si on ne peut pas le faire par le biais d'une balise, alors on peut utiliser un rôle et ses attributs ARIA associés.

Rappel : “No ARIA is better than bad ARIA”

ARIA doit rester un complément à un HTML sémantique et robuste ! Ne l'utiliser que lorsqu'il représente un réel intérêt pour les utilisateurs. Il vaut mieux utiliser peu de ARIA de façon réfléchie que beaucoup de façon automatique. Comme nous l'avons vu précédemment, ARIA peut impacter l'expérience de nos utilisateurs d'une façon positive comme négative. C'est donc un outil important à maîtriser pour réaliser des applications plus accessibles et donc plus inclusives.